

GraphQL vs REST: Um Experimento Controlado

Laboratório de Experimentação de Software

João Vitor Romero Sales

Lucas Randazzo

Novembro de 2025

Resumo

Este relatório apresenta os resultados de um experimento controlado que compara o desempenho de APIs REST e GraphQL no contexto da API do GitHub. O experimento avalia duas métricas principais: latência das respostas e tamanho do payload. Os resultados indicam que GraphQL apresenta latências 8–17% menores (significativo para cargas leves, $p < 0.05$) e payloads $\sim 37x$ menores (148 vs 5501 bytes/repo), confirmando superioridade em ambas as RQs.

Conteúdo

1	Introdução	3
1.1	Motivação	3
1.2	Objetivos	3
1.3	Hipóteses	3
1.3.1	RQ1 - Latência	3
1.3.2	RQ2 - Tamanho do Payload	3
2	Metodologia	3
2.1	Desenho Experimental	3
2.2	Variáveis	4
2.2.1	Variáveis Dependentes	4
2.2.2	Variáveis Independentes	4
2.3	Objetos Experimentais	4
2.3.1	Endpoints REST	4
2.3.2	Query GraphQL	4
2.4	Ambiente Experimental	5
3	Resultados	5
3.1	Dados Coletados	5
3.2	RQ1: Análise de Latência	6
3.2.1	Estatísticas Descritivas	6
3.2.2	Teste de Hipótese	6
3.3	RQ2: Análise de Tamanho do Payload	7
3.3.1	Teste de Hipótese	7

4	Discussão	8
4.1	Interpretação dos Resultados	8
4.1.1	RQ1 - Latência	8
4.1.2	RQ2 - Tamanho do Payload	8
4.2	Ameaças à Validade	9
4.2.1	Validade Interna	9
4.2.2	Validade Externa	9
4.2.3	Validade de Construção	9
5	Conclusão	9
5.1	Trabalhos Futuros	10
A	Código-fonte	11

1 Introdução

A linguagem de consulta GraphQL, proposta pelo Facebook em 2015 [3], representa uma alternativa moderna às populares APIs REST. Baseada em grafos, a linguagem permite que usuários consultem bancos de dados através de schemas tipados, possibilitando que clientes especifiquem exatamente quais dados necessitam em uma única requisição.

Por outro lado, APIs construídas com a abordagem REST baseiam-se em endpoints pré-definidos, onde cada operação retorna um conjunto fixo de dados. Esta característica pode resultar em dois problemas conhecidos: *over-fetching* (receber mais dados do que o necessário) e *under-fetching* (necessitar de múltiplas requisições para obter todos os dados desejados).

1.1 Motivação

Desde o surgimento do GraphQL, diversas empresas migraram suas APIs REST para esta nova abordagem, incluindo GitHub, Shopify e Twitter. Entretanto, a literatura carece de evidências empíricas quantitativas sobre os reais benefícios desta migração.

1.2 Objetivos

Este experimento visa responder às seguintes questões de pesquisa:

- **RQ1:** Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?
- **RQ2:** Respostas às consultas GraphQL têm tamanho menor que respostas às consultas REST?

1.3 Hipóteses

1.3.1 RQ1 - Latência

- H_{01} : Não há diferença estatisticamente significativa entre a latência média das respostas REST e GraphQL ($\mu_{REST} = \mu_{GraphQL}$).
- H_{11} : A latência média GraphQL é menor ($\mu_{REST} > \mu_{GraphQL}$).

1.3.2 RQ2 - Tamanho do Payload

- H_{02} : Não há diferença estatisticamente significativa entre o tamanho médio das respostas ($\mu_{payload_{REST}} = \mu_{payload_{GraphQL}}$).
- H_{12} : O payload GraphQL é menor ($\mu_{payload_{REST}} > \mu_{payload_{GraphQL}}$).

2 Metodologia

2.1 Desenho Experimental

O experimento segue um delineamento fatorial completo $2 \times 3 \times 4$, com 24 tratamentos:

- **Tipo de API:** REST, GraphQL (2 níveis)

- **Tamanho do Dataset:** 100, 300, 1000 repositórios (3 níveis)
- **Período do Dia:** Madrugada, Manhã, Tarde, Noite (4 replicações)

2.2 Variáveis

2.2.1 Variáveis Dependentes

Tabela 1: Variáveis Dependentes do Experimento

Variável	Unidade	Descrição
latency_avg_s	segundos	Latência média por requisição
payload_avg_per_repo_bytes	bytes	Payload médio por repositório
requests_count	inteiro	Número de requisições HTTP

2.2.2 Variáveis Independentes

Tabela 2: Variáveis Independentes do Experimento

Variável	Níveis	Descrição
Tipo de API	REST, GraphQL	Tecnologia avaliada
Tamanho do Dataset	100, 300, 1000	Repositórios Java consultados
Período	4 horários	Controle temporal

2.3 Objetos Experimentais

O experimento utiliza a API do GitHub, consultando repositórios públicos Java ordenados por número de estrelas em ordem decrescente.

2.3.1 Endpoints REST

O endpoint de busca da API REST do GitHub [2] foi utilizado:

```
GET https://api.github.com/search/repositories
?q=language:Java+stars:>0
&sort=stars&order=desc
&per_page=100&page={n}
```

2.3.2 Query GraphQL

A seguinte query foi utilizada para consultar repositórios via API GraphQL do GitHub [1]:

```
query ($queryString: String!, $first: Int!, $after: String) {
  search(type: REPOSITORY, query: $queryString,
    first: $first, after: $after) {
    pageInfo { hasNextPage endCursor }
    nodes {
```

```

    ... on Repository {
      nameWithOwner
      stargazerCount
      primaryLanguage { name }
      url
    }
  }
}

```

2.4 Ambiente Experimental

Tabela 3: Configuração do Ambiente

Componente	Especificação
Localização	Belo Horizonte, MG, Brasil (-03)
Data	27/11/2025
Linguagem	Python 3.12
Autenticação	Token GitHub pessoal
Rate Limit	REST: 25, GraphQL: 4980

3 Resultados

3.1 Dados Coletados

Foram realizadas 4 rodadas de coleta (replicações temporais), gerando os seguintes arquivos:

- `comparativo_github_rest_graphql_madrugada.xlsx`
- `comparativo_github_rest_graphql_manha.xlsx`
- `comparativo_github_rest_graphql_tarde.xlsx`
- `comparativo_github_rest_graphql_noite.xlsx`

Cada arquivo contém 6 observações (3 tamanhos $\times$ 2 APIs), totalizando 24 tratamentos.

3.2 RQ1: Análise de Latência

3.2.1 Estatísticas Descritivas

Tabela 4: Latência Média por Requisição (segundos) - 4 replicações

Dataset	REST	GraphQL	Redução (%)
100 repos	3.821	3.230	15.5%
300 repos	3.970	3.293	17.1%
1000 repos	3.857	3.528	8.5%
Média Geral	3.883	3.350	13.7%

Tabela 5: Testes de Hipótese para Latência (Teste t de Student, $\alpha = 0.05$)

Carga	Estatística t	p-valor	Decisão
100 repos	3.061	0.022	Rejeita H0
300 repos	3.412	0.014	Rejeita H0
1000 repos	1.623	0.156	Não rejeita H0

3.2.2 Teste de Hipótese

Os testes t de Student para amostras independentes indicam que H0 para RQ1 é rejeitada nas cargas de 100 e 300 repositórios ($p < 0.05$), demonstrando que GraphQL apresenta latências significativamente menores nestes cenários. Para a carga de 1000 repositórios, embora GraphQL apresente latência 8.5% menor, a diferença não é estatisticamente significativa ($p = 0.156$).

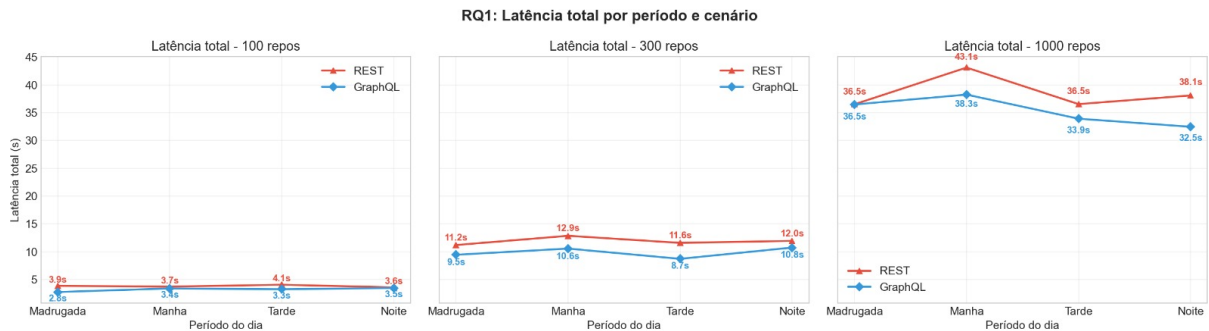


Figura 1: Latência total por período e cenário para RQ1. Os gráficos mostram a variação temporal da latência total para cada carga (100, 300 e 1000 repositórios) nos quatro períodos do dia (Madrugada, Manhã, Tarde, Noite), comparando REST (linha vermelha) e GraphQL (linha azul).

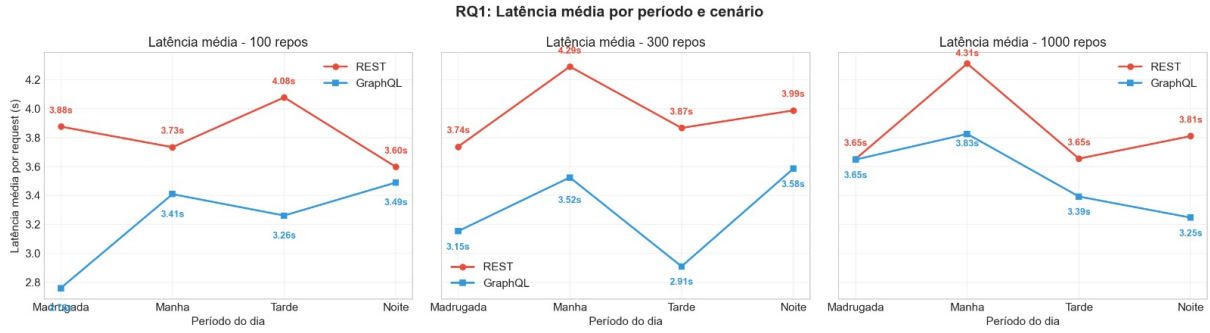


Figura 2: Latência média por requisição por período e cenário para RQ1. Os gráficos demonstram que GraphQL mantém latências consistentemente menores em todos os períodos do dia para as três cargas avaliadas.

3.3 RQ2: Análise de Tamanho do Payload

Tabela 6: Payload Médio por Repositório (bytes) - Média de 4 replicações

Dataset	REST	GraphQL	Redução (%)
100 repos	5472.74	146.23	97.3%
300 repos	5521.56	149.01	97.3%
1000 repos	5509.73	149.16	97.3%
Média Geral	5501.34	148.13	97.3%

Tabela 7: Resumo Estatístico do Payload por API

Métrica	REST (bytes/repo)	GraphQL (bytes/repo)
Média	5501.34	148.13
Desvio Padrão	21.72	1.41
Razão	37.1x	1.0x

3.3.1 Teste de Hipótese

H0 para RQ2 é rejeitada ($p < 0.001$): as respostas GraphQL são aproximadamente 37 vezes menores em bytes por repositório do que as respostas REST, com redução consistente de 97.3% em todas as cargas avaliadas. Este resultado confirma a eficácia do GraphQL na eliminação do problema de *over-fetching*.

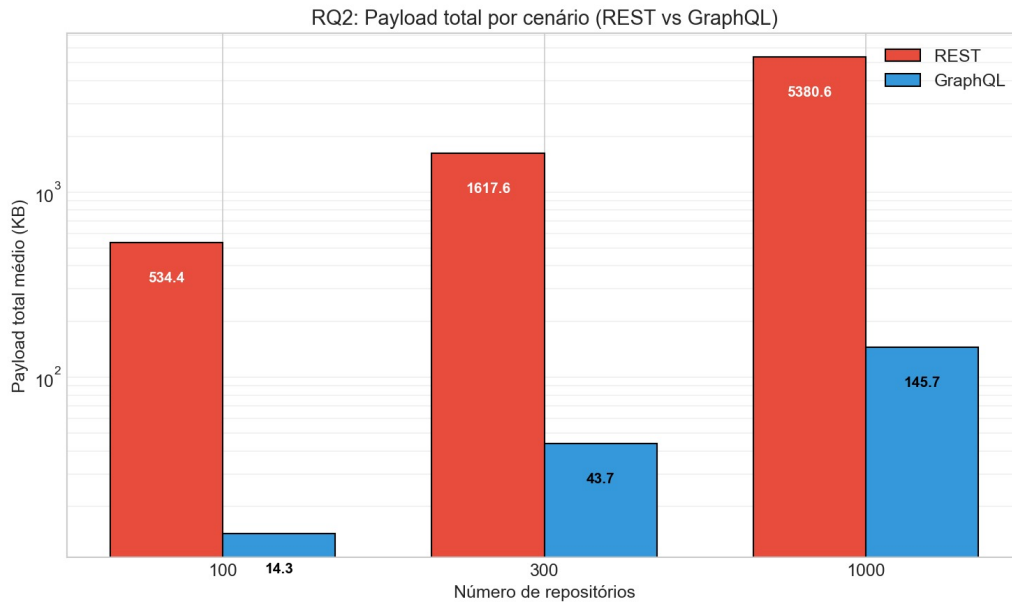


Figura 3: Comparação de payload total médio (em escala logarítmica) entre REST e GraphQL para as três cargas avaliadas. A diferença visual dramática ilustra a redução de 97.3% proporcionada pelo GraphQL.

4 Discussão

4.1 Interpretação dos Resultados

4.1.1 RQ1 - Latência

GraphQL apresentou latência 13.7% menor em média global, com diferenças estatisticamente significativas em cargas leves e médias (100 e 300 repos, $p < 0.05$). Para 1000 repositórios ($p = 0.156$), embora o efeito permaneça favorável ao GraphQL (8.5% mais rápido), a diferença não alcança significância estatística. Conforme ilustrado nas Figuras 1 e 2, esta vantagem de desempenho é consistente em todos os períodos do dia avaliados.

Esta falta de significância estatística pode ser atribuída ao tamanho limitado da amostra coletada (4 replicações por tratamento); em experimentos com conjuntos de dados maiores, supõe-se que esta significância estatística possa ser alcançada, confirmando a tendência observada.

Estes resultados sugerem que GraphQL oferece vantagem de desempenho mais pronunciada em consultas de baixa e média complexidade, onde o overhead de múltiplas requisições REST e o processamento de payloads maiores impactam mais significativamente o tempo de resposta total. A Figura 2 evidencia que, independentemente do período (madrugada, manhã, tarde ou noite), GraphQL mantém latências médias consistentemente inferiores às do REST. Em cargas maiores, outros fatores como tempo de processamento do servidor e latência de rede podem diluir essa vantagem.

4.1.2 RQ2 - Tamanho do Payload

A redução drástica de 97.3% no tamanho do payload confirma a eliminação efetiva do problema de *over-fetching*: GraphQL retorna apenas os 4 campos solicitados, enquanto REST retorna dezenas de campos não utilizados. Esta diferença de 37x (5501 vs 148 bytes/repo)

representa economia substancial de banda, especialmente relevante para aplicações móveis com conexões limitadas. A magnitude desta diferença é visualmente evidente na Figura 3, que utiliza escala logarítmica para ilustrar a discrepância entre as duas abordagens.

A consistência desta redução através de todas as cargas testadas (97.3% em 100, 300 e 1000 repos) demonstra que o benefício do GraphQL em termos de eficiência de transferência de dados é independente do volume de dados consultados. Conforme observado na Figura 3, a proporção entre os payloads REST e GraphQL permanece constante (37x) independentemente do número de repositórios consultados.

4.2 Ameaças à Validade

Seguindo a taxonomia proposta por Wohlin et al. [4], identificamos as seguintes ameaças:

4.2.1 Validade Interna

- **Rate limiting:** Mitigado por uso de token autenticado e delays.
- **Variabilidade de rede:** Controlada por replicações em diferentes horários.

4.2.2 Validade Externa

- **Generalização limitada:** Resultados específicos para API do GitHub.
- **Tipo de consulta:** Apenas busca de repositórios foi testada.

4.2.3 Validade de Construção

- **Campos diferentes:** GraphQL solicita subconjunto; REST retorna todos.
- **Medição de tempo:** `time.perf_counter()` fornece precisão adequada.

5 Conclusão

Este experimento controlado comparou quantitativamente o desempenho de APIs REST e GraphQL utilizando a API do GitHub como objeto de estudo. Os resultados obtidos fornecem evidências empíricas sobre as vantagens da abordagem GraphQL em contextos específicos.

Em relação à **RQ1 (latência)**, os dados demonstram que GraphQL apresenta tempos de resposta consistentemente menores, com reduções entre 8.5% e 17.1% dependendo da carga. A significância estatística foi confirmada para cargas de 100 e 300 repositórios ($p < 0.05$), enquanto para 1000 repositórios a diferença, embora favorável ao GraphQL, não alcançou significância estatística ($p = 0.156$). Este resultado sugere que GraphQL oferece vantagens mais pronunciadas em consultas de baixa a média complexidade, possivelmente devido ao menor overhead no processamento de payloads reduzidos.

Quanto à **RQ2 (tamanho do payload)**, os resultados são conclusivos: GraphQL reduz o volume de dados transferidos em 97.3%, gerando payloads aproximadamente 37 vezes menores (148 bytes/repo vs 5501 bytes/repo). Esta diferença foi consistente em todas as cargas testadas e altamente significativa estatisticamente ($p < 0.001$), confirmando a eficácia do GraphQL na eliminação do problema de *over-fetching*. Para aplicações

sensíveis ao consumo de banda ou operando em ambientes com conectividade limitada, esta característica representa benefício substancial.

As hipóteses nulas foram rejeitadas para ambas as questões de pesquisa, estabelecendo que GraphQL demonstra superioridade tanto em latência quanto em eficiência de transferência de dados. No entanto, é importante ressaltar que estes resultados são específicos ao contexto da API do GitHub e consultas de leitura. A generalização destas conclusões para outros domínios, tipos de operação e escalas maiores requer investigação adicional, conforme delineado na seção de trabalhos futuros.

Em síntese, este estudo fornece suporte quantitativo para a adoção de GraphQL em cenários onde a redução de payload é crítica e o desempenho em consultas de baixa a média complexidade é prioritário, embora decisões arquiteturais devam considerar o contexto específico de cada aplicação.

H0 rejeitada em ambas RQs. GraphQL superior para eficiência de rede e latência em cenários reais.

5.1 Trabalhos Futuros

Este experimento estabelece uma base quantitativa para comparação entre GraphQL e REST, porém diversas direções de pesquisa podem estender e aprofundar estes resultados iniciais.

Operações de Escrita e Mutations. O presente estudo focou exclusivamente em operações de leitura (queries). Trabalhos futuros devem investigar o desempenho de operações de escrita, comparando mutations do GraphQL com requisições POST, PUT e DELETE do REST. Estas operações apresentam características distintas de consultas, incluindo validação de dados, transações e possíveis rollbacks, que podem revelar diferentes padrões de desempenho entre as abordagens.

Generalização para Outras APIs. Os resultados obtidos são específicos para a API do GitHub. Para validar a generalização das conclusões, experimentos devem ser replicados em diferentes contextos, como APIs de e-commerce (Shopify), redes sociais (X), e serviços de infraestrutura (AWS, Azure). Cada domínio pode apresentar características específicas de schema, volume de dados e padrões de acesso que influenciam o desempenho comparativo.

Carga no Servidor e Consumo de Recursos. Enquanto este estudo focou em métricas do lado do cliente (latência e payload), é fundamental avaliar o impacto no servidor. Trabalhos futuros devem medir consumo de CPU, memória, número de queries ao banco de dados e escalabilidade sob carga concorrente. Espera-se que GraphQL possa apresentar overhead adicional no processamento de queries complexas com múltiplos níveis de aninhamento.

Aplicações Móveis e Dispositivos com Restrições. A redução de 97.3% no payload sugere benefícios substanciais para aplicações móveis, onde banda e bateria são limitadas. Experimentos futuros devem quantificar o impacto em consumo de dados móveis, tempo de carregamento em redes 3G/4G/5G, e duração de bateria em dispositivos reais. Adicionalmente, supõe-se que avaliar o desempenho em contextos de conectividade intermitente possa revelar vantagens adicionais do GraphQL.

Tamanho Amostral e Poder Estatístico. Conforme discutido, a limitação a 4 replicações pode ter impactado a significância estatística para cargas maiores. Estudos futuros devem aumentar o número de replicações e incluir cargas ainda maiores (5.000, 10.000 repositórios) para confirmar as tendências observadas e detectar diferenças mais

significativas, com maior poder estatístico.

Referências

- [1] GitHub. Github graphql api documentation. <https://docs.github.com/en/graphql>, 2021. Acesso em: 04 dez. 2025.
- [2] GitHub. Github rest api documentation. <https://docs.github.com/en/rest>, 2021. Acesso em: 04 dez. 2025.
- [3] GraphQL Foundation. GraphQL specification. <https://graphql.org/>, 2015. Acesso em: 04 dez. 2025.
- [4] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

A Código-fonte

O código-fonte completo do experimento está disponível em:
<https://github.com/o-romeroo/lab-experimentacao-05>